1 PROOF OF LEMMA

LEMMA 2. *Given a streaming dataset $\mathcal{S}$, let R_i and R_{i+1} (= $R_i \cup \{p(o, o')\}$) be the two result sets to KCP queries corresponding to $k = i$ and $k = (i+1)$, respectively. It is guaranteed that if $o \notin R_i.obj \vee o' \notin R_i.obj$, $p(o, o') \in \mathcal{N} - R_i$.*

PROOF. If $o \notin R_i.obj$, $p(o, NN(o)) \notin R_i$. Based on the fact that $o \in R_{i+1}.obj$, o' must be $NN(o)$. Follow the same logic, if $o' \notin R_i.obj$, $NN(o')$ must be o. Therefore, $p(o, o')$ belongs to $\mathcal{N} - R_i$. The proof completes. □

LEMMA 3. *Let $e_i.O$ be the set of objects in e_i, $c_{max}(\eta_{\frac{1}{2}})$ be the max cube constructed based on objects in $e_i.O$, $e_i.c$ be the standard cube with minimal side-length that bounds objects in $e_i.O$, and c be any standard cube in the space of $e_i.c$ not covering $c_{max}(\eta_{\frac{1}{2}})$. It is guaranteed that both c and $c_{max}(\eta_{\frac{1}{2}})$ contain* no *more than $\frac{|e_i.O|}{2}$ objects.*

PROOF. Since neither $\eta_{\frac{1}{4}}$ nor $\eta_{\frac{3}{4}}$ is within $c_{max}(\eta_{\frac{1}{2}})$, at least $\frac{|e_i.O|}{2}$ objects are out of $c_{max}(\eta_{\frac{1}{2}})$. Thus, $c_{max}(\eta_{\frac{1}{2}})$ contains no more than $\frac{|e_i.O|}{2}$ objects. For each standard cube c that is within the space $e_i.c$ and meanwhile does not contain $c_{max}(\eta_{\frac{1}{2}})$, it does not contain $\eta_{\frac{1}{2}}$. Accordingly, $\exists i \in [1, d]$ such that i) $c[i] > \eta_{\frac{1}{2}}[i]$ or ii) $c[i] + |c| < \eta_{\frac{1}{2}}[i]$. For both case (i) and case (ii), there are at least $\frac{|e_i.O|}{2}$ objects are outside c. Thus, c also contains no more than $\frac{|e_i.O|}{2}$ objects. □

LEMMA 4. *Given $C_\delta = \{c \in C | |c| \leq \delta\}$, there must exist a standard cube with side length 2δ that bounds all the cubes in C_δ.*

PROOF. All the cubes in C, the super set of C_δ, are formed by mapping rules (1) or (2). Accordingly, we partition cubes in C_δ into two disjoint subsets, C_δ^1 containing the cubes generated by rule (1) and C_δ^2 containing cubes generated by rule (2). For cubes c_i in C_δ^1, c_i is a standard cube of side length $\frac{|c_i'|}{2}$, with standard cube c_i' being the minimal standard cube that bounds both o_i and $c_{max}(\eta_{\frac{1}{2}})$ and o_i is located outside $c_{max}(\eta_{\frac{1}{2}})$. In other words, $|c_i| = \frac{|c_i'|}{2} \geq |c_{max}(\eta_{\frac{1}{2}})|$. We form a complementary set $C_\delta^{1'}$ that captures all the standard cube c_i' corresponding to $c_i \in C_\delta^1$. For cubes c_j in C_δ^2, it is guaranteed $c_j = c_{max}(\eta_{\frac{1}{2}})$.

On the other hand, given any two standard cubes c_i and c_j with $|c_i| \geq |c_j|$ and meanwhile both covering $c_{max}(\eta_{\frac{1}{2}})$, it is guaranteed that c_i fully covers c_j. Consequently, since all the cubes in $C_\delta^{1'}$ cover $c_{max}(\eta_{\frac{1}{2}})$, they (and hence all the cubes in C_δ^1) are fully bounded by the standard cube c_j' in $C_\delta^{1'}$ with the largest side-length (i.e., 2δ). This standard cube c_j' covers all cubes in C_δ^2 too. In other words, the standard cube c_j' with side length 2δ covers all the cubes in C_δ. □

THEOREM 1. *Given a non-leaf node e of QC-Tree, let Child(e) represent all its child nodes. If there is one child node $e_i \in Child(e)$ having more than $\frac{e.n}{2}$ objects, it is guaranteed that all the child nodes of e_i have no more than $\frac{e.n}{2}$ objects, i.e., if $\exists e_i \in Child(e)$, $e_i.n \geq \frac{e.n}{2}$, then $\forall e' \in Child(e_i)$, $e'.n \leq \frac{e.n}{2}$.*

PROOF SKETCH: when e is partitioned into 4 quadrants (e_1 to e_4), e_5 is introduced only if the number of objects located inside a quadrant (say e_4) is larger than $\frac{e.n}{2}$. Consequently, we only need to prove that all the child nodes of e_4 and e_5 have no more than $\frac{e.n}{2}$ objects. A $e_5.c$ is the corresponding to c_ξ and $|c_\xi|$ =min($2s_{me}$, $\frac{|e_4.c|}{2}$), c_ξ may contain more objects than e_4 as at least half of objects bounded by e_4 are contained in c_ξ. As each sub-space of c_ξ with side-length $\frac{|c_\xi|}{2}$ contain no more than $\frac{n'}{2}$ ($n' = e_4.n + e_5.n < e.n$) based on the construction of c_ξ, it must contain less than $\frac{e.n}{2}$ objects.

THEOREM 2. *Once all the range queries corresponding to objects in $\mathcal{S}$ are processed, $\forall o \in P_i$, if $o.NN$ is NULL, we have $m_i < \mathsf{D}(o)$; otherwise, $o.NN = NN(o)$.*

PROOF. Given $o \in P_i$, if $o.NN$ is still $NULL$ after processing all the range queries, it is guaranteed that $\mathsf{D}(o) > m_i$ though we have not found $NN(o)$. This can be proved by contradiction. We assume $\mathsf{D}(o) \leq m_i$, and hence there must be an object o' such that $\mathsf{D}(o, o') \leq m_i$. If $o' \in \cup_{j \in [1,i]} P_j$, it will be retrieved by the i range queries issued based on o, as the search ranges of those i queries are no smaller than m_i. Otherwise, $\exists j \in [i+1, m]$ such that $o' \in P_j$. The range query $q_i(o', m_i, P_i)$ issued based on o' on objects in P_i will bound object o and hence update $o.NN$. Given the fact that $o.NN$ is still $NULL$, our assumption $\mathsf{D}(o, o') \leq m_i$ is invalid and hence $\mathsf{D}(o) > m_i$. If $o.NN$ is not $NULL$, it means that we have found a real object with $\mathsf{D}(o, o.NN)) \leq m_i$. It is guaranteed that $o.NN = NN(o)$, which can be easily proved following the logic we use to prove the above case. □

THEOREM 3. *Given an object o to be remapped/inserted to a partition, assume partition P_i is the first partition such that $C_i^{q_i} = \emptyset$ or $\sum_{c \in C_i^{q_i}} c.n = 0$ and o is inserted into $P_j (1 < j < m)$. It is guaranteed $m_j \leq \mathsf{D}(o) \leq \mu m_{j-1}$ with μ bounded by $(1 + \sqrt{d})$.*

PROOF. As stated previously, when we insert object o to a proper partition, there are in total two cases when we find a partition P_i ($i < m$) such that $C_i^{q_i} = \emptyset$ or $\sum_{c \in C_i^{q_i}} c.n = 0$. To facilitate the following proof, let o' and o'' be the object in the first i partitions $\cup_{l=1}^{i} P_l$ and next $(m - i)$ partitions $\cup_{l=i+1}^{m} P_l$ having the shortest distance to o respectively. In other words, it is guarantee that i) $\forall obj \in \cup_{l=1}^{i} P_l$, $\mathsf{D}(obj, o) \geq D(o', o)$ and ii) $\forall obj' \in \cup_{l=i+1}^{m} P_l$, $\mathsf{D}(obj', o) \geq \mathsf{D}(o'', o) \geq m_i$.

In the first case, $\mathsf{D}(o, o') < m_i$ and o is inserted into a partition P_j ($j > i$) such that $m_j \leq \mathsf{D}(o, o') < m_{j-1}$. Under this case, as $D(o'', o) \geq m_i$, o' must be $NN(o)$,i.e., $\mathsf{D}(o)$=$\mathsf{D}(o, o')$. We have $m_j \leq \mathsf{D}(o, o')(=\mathsf{D}(o)) < m_{j-1}$ with $\mu = 1$. In the second case, $\mathsf{D}(o, o') \geq m_i$, $C_i^{q_i} = \emptyset$ or $\sum_{c \in C_i^{q_i}} c.n = 0$. o is inserted into P_i. It's guaranteed that $\mathsf{D}(o) \geq m_i$. As the search for partition P stops at P_i, there is at least one standard cube c of C_{i-1} associated with partition P_{i-1} that overlaps with the search range of q_{i-1} and $c.n > 0$. As $c.n > 0$, there is at least one object o''' in the subsequent partitions $\cup_{l=i}^{m} P_l$ fallen with c. As c overlaps with the search region, the minimal distance between o and c is no larger than m_{i-1}. Accordingly, the maximal distance between o and c is no larger than $m_{i-1}(1 + \sqrt{d})$. Therefore, we have $\mathsf{D}(o, o''') \leq (1 + \sqrt{d}) m_{i-1}$, i.e., $m_i \leq \mathsf{D}(o) \leq (1 + \sqrt{d}) m_{i-1}$ with $\mu = 1 + \sqrt{d}$ and $j = i$. □

2 THE LOCAL RE-PARTITION VIA GUARD

As we mention previously, the split and merge of the last partition P_m, in an extreme case, might happen frequently which incurs a very high maintenance cost of $\tau-$DLBP partition. In order to lower down the update cost and mitigate the risk of frequent change of P_m, we propose a new data structure namely *Guard*, formally defined below. Note, $P_i.\theta$ refers to the corresponding $g(\tau)$ used by TNNS when P_i is the last partition that covers all the objects in TNNS; $P_i.O$ refers to the set of objects in P_i with scores between $(\mathsf{LB}(P_i), P_i.\theta]$ (i.e., those objects in P_i but not in TNNS); $P_i.P_{nn}$ refers to the set of corresponding NN pairs formed by objects in $P_i.O$; and $P_i.t$ refers to the time stamp when P_i is split and changes from the last partition to the second last partition.

DEFINITION 5. **Guard**. *A guard, expressed as a tuple $\langle \theta, O, P_{nn}, t\rangle$, is associated with a partition P_i, iif P_i is used to be the last partition of $\tau-$DLBP at a time stamp within the window $[t_{cur} - |P_i.O|, t_{cur}]$ with t_{cur} referring to the current time stamp.*

Let $\mathcal{S}$ be partitioned into $\{P_1, P_2, \cdots, P_m, P_{m+1}\}$ and P_m be a partition associated with a guard structure. When a few objects in P_{m+1} expire, local re-partition might be triggered because of condition i) or iii). For both cases, we need to find the scores of objects in P_{m+1} and P_m. Instead of relying on range queries to find objects' scores, we could utilize $P_m.P_{nn}$ maintained by the guard structure. In the following, we will discuss guard maintenance under data stream.

Guard forming and deconstructing. A guard is formed at the moment (say t_m) the last partition P_m of $\tau-$DLBP is split into P_m and P_{m+1}. In order to efficiently maintain the guard of the partition P_m, side-length of cubes in the corresponding cube set C_m is set to $P_i.\theta$ other than $\mathsf{LB}(P_i)$. Along with $\mathcal{S}$ updates, the guard associated with P_m is deconstructed at the moment $t_m + |P_m.O|$, assuming the total number of partitions is larger than m from t_m to $t_m + |P_m.O|$. Accordingly, side-length of cubes in C_m is updated to $\mathsf{LB}(P_m)$. The intuition behind it is if the last partition is not frequently split *or* merged, even we do not maintain the *Guard* structure, the amortized local re-partition cost is not high. So, after $|P_m.O|$ updates, if the current P_m is still not used for forming TNNS, we deconstruct the *Guard* structure so as to reduce the space cost.

Guard maintenance under data stream. Given a dataset $\mathcal{S}$ partitioned into $\{P_1, P_2, \ldots, P_r\}$, let P_i be the partition with smallest index i associated with a guard structure. When a new object o arrives in $\mathcal{S}$, we process o based on the algorithm presented in Section 4.3. When we evaluate partition from P_1 to P_{i-1}, if there is an object o' in $P_j (j < i)$ having distance to o between $\mathsf{LB}(P_i)$ and $P_i.\theta$, we update guard of P_i based on o'. When evaluating P_i, we submit a range query $q(o, P_i.\theta, P_i)$, and then update guard of P_i based on $R_i(o)$. If the search is terminated after accessing P_i, $\mathsf{D}(o)$ is also bounded. The proof is similar with that is stated Theorem 3, we skip the details. When an object o_{exp} expires from a partition P_i that is associated with a guard structure, if $o_{exp}.d > P_i.\theta$, we remove it from P_i. Otherwise, we process its reverse nearest neighbours following the logic of processing expired objects in $T.obj$,i.e., updating guard of P_i based on distance among o_{exp}'reverse nearest neighbours and their new nearest neighbours.

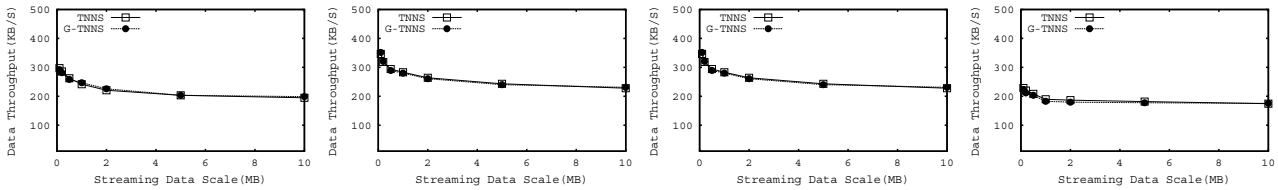
3 COST ANALYSIS

QC-Tree Construction Cost Analysis. Given a QC-Tree $\mathcal{I}$, let e be a non-leaf node. When we form its child nodes, it first requires $O(d \cdot e.n)$ to partition the space (and objects) into 2^d quadrants. If the construction of e_5 is required (i.e., one of the child node e_i has at least $e.n/2$ objects), it incurs additional running cost to form $\eta_{max}(\frac{1}{2})$ and c_{ξ}. The former requires quantile-search whose running cost is linear to the data set scale with hidden constant d; the latter also involves quantile-search with computational cost bounded by $O(e.n)$. Given the fact that there are in total $e.n$ objects in e, the running cost of forming its child nodes per object is $O(d)$. As the height of QC-Tree is bounded by $O(\log n)$, there are at most $O(\log n)$ nodes containing any given object o. Accordingly, the QC-Tree construction running cost is bounded by $O(d \cdot n \cdot \log n)$. Once QC-Tree is formed, L Cube of each object is formed accordingly. Thus, the overall running cost of L Cube construction is $O(d \cdot n \cdot \log n)$.

$\tau-$DLBP Construction Cost Analysis. The $\tau-$DLBP construction involves three steps, including L Cube-based partition, score lower-bound calculation and refinement. For the first step, the running cost of constructing L Cubes is bounded by $O(dn \log n)$. As we use quantile search based algorithm to form the partition, the running cost of L Cube-based partition is bounded by $O(n)$. For the second step, each object $o \in P_i$ submits i range queries. When o searches on P_j $(1 \leq j \leq i)$, the search region overlaps with 3^d cubes. If $i, j \neq m$, each cube contains at most one object. The running cost spent on one range query is bounded by $O(3^d)$, and hence the total running cost spent on objects in P_i is bounded by $O(3^d i|P_i|)$. Otherwise (*i.e.*, $i = j = m$), in an extreme case, as side-length of 2τ objects' L Cubes is no larger than $|c^{2\tau}|$, such objects may be packed in *one* cube with side-length m_m and the range query cost per object is bounded by $O(3^d + \tau)$. In normal cases, at most 3^d objects in P_m are located inside neighbour cubes of c, and hence the running cost per object is also $O(3^d + \tau)$. Other objects in P_m spend $O(3^d)$ running cost. Totally, running cost spent on range queries is bounded by $O(\sum_{i=1}^{i=m} 3^d i|P_i| + 3^d(|P_m| - 3^d) + 3^d(3^d + \tau) + \tau(\tau + 3^d))$. As $|P_i| \leq \frac{n}{2^i}$, the running cost equals to $O(\sum_{i=1}^{i=m} i\frac{n}{2^i} + \tau^2)$, which is $O(3^d n + \tau^2)$. For the running cost of refinement, it is $O(n)$. Similarly, the running cost of forming cube sets is also $O(n)$. Thus, the total $\tau-$DLBP construction cost is $O(3^d n + \tau^2 + dn \log n)$.

Local Re-partition Cost Analysis. We first analyze the running cost of local re-partition under condition (i), where we assume that the algorithm is terminated after evaluating objects in $P_i \cup P_m$. The running cost includes: (i) local-L Cube maintenance running cost; and (ii) range queries cost. For the former one, as local-L Cube of every object is formed at most once after $\mathcal{S}$ updates n times and $|P_m| < |P_i|$, the running cost is bounded by $O(d|P_i \cup P_m| \log |P_i \cup P_m|)$. As $|P_i \cup P_m|$ is bounded by n, this part of running cost is bounded by $O(dn \log n)$,i.e., $O(d \log n)$ per object.

For the later one, after we access objects in $P_i \cup P_m$, each object $o \in P_j (j > i)$ goes through two stages. In stage one, we perform local re-partition using objects in the original partitions from P_m to P_{i+1}. During this stage, each object in $P_j (j > i)$ in total submits $j - i$ range queries, where the search range of each query overlaps up to 3^d standard cubes. Therefore, the cube accessing cost spent on each object is $O((j - i)3^d)$. Besides, as the algorithm is not terminated after accessing P_{i+1}, it means the total number of objects



(a) Stock ($s = 0.1\% \times n$) (b) Trip ($s = 0.1\% \times n$) (c) Uniform ($s = 0.1\% \times n$) (d) Normal ($s = 0.1\% \times n$)

Figure 4: Data throughput comparison of TNNS under Guard under different data sets under different n

contained in query regions of all queries is bounded by $O(\tau)$. As $|P_j|$ is excepted to be $\frac{|P_j|}{2^{j-i}}$, the overall running cost under stage one is $O(\sum_{j=i+1}^{m} 3^d(j-i-1)|P_j|+\tau)$, which is $O(3^d|P_i|+\tau^2)$. In stage two, we use objects in the current P_m and P_i for local re-partition, with objects in $P_i \cup P_m$ mapped into cubes with side-length $\mathsf{LB}(P_{i-1})$. If the number of local-L Cubes with side-length no larger than $\frac{m_{i-1}}{2\sqrt{d}}$ is at least 2τ, it applies $\tau-$DLBP construction algorithm to form the new partitions with running cost bounded by $O(|P_i|3^d + \tau^2)$. Otherwise (i.e., the number is small than 2τ), $2\tau - 1$ objects may fall within *one* cube c with side-length m_{i-1} in the worst case. The range query spent on each of them is bounded by $O(3^d + \tau)$. In the other cases, at most 3^d objects in P_i are located inside the neighbour cubes of c that o is located, and the running cost spent on each of them is $O(3^d + \tau)$. Other objects (i.e., those located outside the neighbor cubes of c) in P_i spend $O(3^d)$ running cost. Accordingly, the total running cost is bounded by $O(|P_i|3^d + \tau^2)$. As local re-partition under case (i) is triggered by expiration of objects in $P_m \cup P_i$, and we use the structure guard to mitigate the risk of frequent change of P_m, this part of cost is bounded by $O(3^d)$ per update. In brief, regardless of the number of such local L Cubes, the cost spent on each local re-partition under condition i) is bounded by $O(\tau + 3^d + d \log n)$ per object.

In the following, we analyze the cost of local re-partition under conditions (ii) and (iii). Under condition (ii), when P_m is split, as we have computed nearest neighbours for all the objects in P_m, we can use median search algorithm to partition the original P_m into P_m and P_{m+1} with cost bounded by $O(|P_m|)$. Under condition (iii), assuming we merge P_i into P_{i+1}, the running cost is bounded by $O(|P_i| + |P_{i+1}|)$, i.e., $O(1)$ per object.

The incremental maintenance cost. The arrival of new object(s) triggers in total three operations, as detailed below. First, when an object o flows into S, we assume it is finally inserted into $P_i (j < m)$, with the running cost bounded by $O(3^d j)$. However, if o is inserted into P_m, the running cost is $O(3^d \log \frac{n}{\tau} + \tau)$. Second, the arrival of o reduces the score of another o'. Say $m_{i'} \leq D(o') < m_{i'-1}$, and object o' is moved from P_i to $P_{i'}$ with running cost being $O(i' - i)$. Third, local re-partition under condition (ii) or (iii) happens. As discussed before, each object spends $O(1)$ running cost per time. After n newly arrived objects, assuming all objects are inserted into the last partition P_m, the total running cost is bounded by $O(3^d n \log \frac{n}{\tau} + n\tau)$,i.e.,$O(3^d \log \frac{n}{\tau} + \tau)$ per object.

Similarly, the expiry of an object triggers three different operations too. First, when an object $o_{exp} \in P_i$ expires from $\mathcal{S}$, it is removed from P_i. At the same time, for each cube c with side-length $m_j (j < i)$ that bounds o_{exp}, we set $c.n$ to $c.n - 1$. Second, it may trigger local-L Cubes maintenance with cost bounded by $O(d \log n)$. Third, it may trigger local re-partition under condition (i) and condition (iii). As discussed before, after n updates, the local L cube of each object is formed once at most with running cost bounded by $O(d \log n)$. Assuming all deletions are from the last partition, the total local re-partition running cost under case (i) is bounded by $O(3^d \log \frac{n}{\tau} + d \log n + n\tau)$ after n updates. As the running cost of local re-partition under condition (iii) is $O(1)$ per object, the total running cost of maintaining $\tau-$DLBP is bounded by $O(3^d \log \frac{n}{\tau} + \tau + d \log n)$ per object.

We now analyze the space cost of $\tau-$DLBP. Let $C_i (i \in [1, m])$ be the set of standard cubes with side-length m_i that bound objects in $\cup_{l=i}^{m} P_l$. In the worst case, each cube bounds one object in $\cup_{l=i+1}^{m} P_l$. Thus, $|C_i|$ is bounded by $\sum_{l=i}^{m} |P_l|$. As $|P_i| > 2|P_{i+1}|$, we have $|P_i| > \sum_{i+1}^{l=m} |P_l| > \frac{\sum_{i}^{l=m} |P_l|}{2}$. Thus, $|C_i|$ is bounded by $2|P_i|$, and $\sum_{i=1}^{m} |C_i|$ is bounded by $2\sum_{i=1}^{m} |P_i|$, which is $O(n)$. Besides, each cube consumes $O(1)$ space cost. The overall space cost of $\tau-$DLBP is bounded by $O(n)$.

4 EXPERIMENT UNDER GUARD

In the following, we supplement a set of experiments, where we evaluate the performance of TNNS under guard, denoted as GTNNS. As the performance of TNNS without guard performs much better than the others, we only compare GTNNS with TNNS.

We report the data throughput of all the algorithms under different n values in Figures 4(a)-4(d). We can observe that the data throughput of GTNNS is roughly as the same as that of TNNS. The reasons are as follows. Firstly, τ-DLBP guarantees that TNNS only maintains a small number of NN pairs, compared with the overall running cost of processing all objects in the streaming data set, the proportion of TNNS maintenance cost is small. Secondly, the frequency of local re-partition is not high. Our propose algorithm guarantees that local re-partition happens only when the distribution of streaming data is drastically changed,i.e., many NN pairs with small scores are formed or deconstructed. Last but not least, the extreme case discussed in Section 4.4 is almost not happened in real applications. Therefore, no matter whether guard is used for maintaining $\tau-$DLBP, we also could efficiently support KCP search.